Zephyr Project

IT-Schulungen.com
Technology & Transformation

Kompakte Netzwerkgrundlagen unter native_sim

Paketweg, Testmodi und TCP-Minimalvertrag

4.08

Fünf Ebenen – fünf Verantwortungen

Vom Fachobjekt bis zum simulierten Netzwerkpfad

01	02	03	04	05
Anwendung	**Socket-API**	**TCP / IP**	**net_if + L2**	**Treiber / Host**
Probe und Fachlogik	Endpunkt und Bytes	Strom, Adresse, Route	Interface und Linkzustand	TAP oder Offload

Nur der Adapter kennt Socket- und Netzwerkdetails; die Sensor-Pipeline bleibt unverändert.

Interface, Adresse, Route und Socket

INTERFACE

Träger des Netzwerkzustands

admin up / oper up

Adressen zugeordnet

Link- und L2-Kontext

ADRESSE + ROUTE

Erreichbarkeit und Weg

lokales Präfix

nächster Hop

Ausgangsinterface

SOCKET

Anwendungsendpunkt

Protokoll + IP + Port

Zustand der Sitzung

kein Ersatz für Interfacezustand

Leitfrage: Welche Ebene ist noch nicht bereit – Interface, IP-Weg oder Anwendungssocket?

TAP und Native Sockets Offload

TESTMODI

```
TAP:      App → Zephyr TCP/IP → L2 → TAP → Host
Offload:  App → Socket-Offload → Linux-Host-Stack
```

01 TAP-Pfad

prüft Zephyrs eigenen TCP/IP-, net_if- und L2-Datenpfad.

02 Offload-Pfad

prüft Anwendung und Socket-Nutzung mit dem Netzwerkstack des Hosts.

Gleiche Socket-API, aber unterschiedliche Tiefe und Aussage des Tests.

TCP ist ein Bytestrom

MINIMALES FRAMING

```
Empfang 1:  "GET"
Empfang 2:  "\nGET\npa"
Empfang 3:  "rtial"

Frames: "GET" · "GET" · Rest "partial"
```

01 Fragmentiert

Ein Request kann über mehrere recv()-Aufrufe verteilt sein.

02 Gebündelt

Ein recv()-Bereich kann mehrere vollständige Frames enthalten.

03 Restdaten

Unvollständige Bytes bleiben bis zum nächsten Empfang erhalten.

Für den Kurstag genügt ein begrenzter Puffer mit Zeilenende als Framegrenze.

Minimaler Socket-Zustand

ZUSTANDSWECHSEL

```
Listener → Session → Daten / Peer-Close / Fehler → Cleanup
```

`Listening-Socket`	nimmt eine Verbindung an und bleibt konzeptionell getrennt
`Session-Socket`	gehört genau zu einem Peer und einem Cleanup-Pfad
`recv() > 0`	Bytes an den Akkumulationspuffer übergeben
`recv() == 0`	geordnetes Peer-Close erkennen
`recv() < 0 / Timeout`	Fehler unterscheiden, Session definiert aufräumen

Listening- und Session-Socket werden nur konzeptionell getrennt – nicht vollständig implementiert.

Netzwerkzustand beobachten

01	`Zephyr: net iface`	Interface, Admin-/Oper-State und zugeordnete Adressen prüfen
02	`Zephyr: net conn`	aktive Verbindungen und lokale beziehungsweise entfernte Endpunkte sehen
03	`Hostwerkzeuge`	TAP mit ip addr; Endpunkt mit ss oder vorbereitetem Client prüfen

Beobachtung benennt die fehlerhafte Ebene, bevor Anwendungscode geändert wird.

TCP bleibt ein Adapter der Sensor-Pipeline

EINGANG

Portable Probe

Sequenz
Datenalter
Qualität
Messwert

ADAPTER

Format + Transport

Probe serialisieren
Frame begrenzen
Socketzustand behandeln

FACHLOGIK

Bleibt unabhängig

```
keine Deskriptoren
keine IP-Adressen
keine Framingdetails
```

Der Transport kann wechseln, ohne Qualitätsmodell oder Sensor-Pipeline zu verändern.

Was heute bewusst nicht vertieft wird

AUSSERHALB DES BLOCKS

`Mehrfachsessions · poll() · komplexe Backpressure · Protokolloptimierung`

MEHRFACHSESSIONS	kein Thread- oder Eventmodell für viele Peers
POLL UND ASYNC	keine Multiplexing- oder Callback-Architektur
BACKPRESSURE	keine dynamische Last- und Pufferoptimierung
PROTOKOLLDETAILS	nur begrenztes Zeilenframing statt Binärformat-Design

Die Reduktion schafft Zeit für Zephyr-spezifische Zustände, Tests und Diagnose.

Lernkontrolle: Pfad und Aussage

KOMPAKTE KARTE

```
Probe → TCP-Adapter
          ↓
Socket-API → TCP/IP → net_if
          ↓
      TAP | Offload
```

01 Erklären

Interface, Adresse, Route, Socket und Bytestrom

02 Einordnen

aktiven Testmodus und seine Aussagegrenze

Ergebnis: Die TCP-Minimalarchitektur ist klar, ohne die Fachlogik an Sockets zu koppeln.